

HeyMax Analytics Project

Documentation

Table of Contents

- Project Overview..... 1
- Features 1
- Setup..... 1
- About the Data..... 1
- Technology Stack 1
 - DBT..... 1
 - DuckDB 2
 - Streamlit..... 2
 - GitHub Deployment..... 2
- Design Considerations..... 2
 - 1. Data Modeling and Transformation Logic..... 2
 - 2. Dashboard Design and Metric Clarity..... 2
 - 3. Code Readability, Structure, and Documentation 2
 - 4. Design Tradeoffs, Testing, and Scaling Strategy 2
- DBT Modeling Strategy..... 2
 - Model Summary..... 3
 - Layering and Its Benefits 3
- Analysis and Insights..... 4
 - Assumptions..... 4
 - Metric Definitions 4
 - Insights 4
 - Follow-up Questions..... 5
- Future Enhancements and Scaling Roadmap 5
 - Additional Enhancements to current dashboard..... 5

Project Overview

This project sets up an end-to-end open-source data and analytics pipeline for HeyMax with an interactive analytics dashboard built using Streamlit, DuckDB, and dbt to analyze user lifecycle metrics over time. It supports monthly, weekly, and daily views with charts, retention triangle tables, KPIs, filters, and LLM-based insights.

Access the live dashboard here: <https://heymax-analytics-dev.streamlit.app/>

Features

-  Toggle between Monthly, Weekly, and Daily metrics
-  Charts for:
 - New, Resurrected, Retained, Churned, and Active Users
 - Stacked bar + line charts (Quick Ratio, Retention Rate)
-  Retention Triangle (monthly, weekly, and daily cohorts)
-  KPI cards for the latest period
-  Filters: date range
-  Ask your data (powered by OpenAI's GPT-4)
-  Download filtered report as CSV

Setup

To setup the project, please refer to the README.md file:

<https://github.com/SANCHIT-GARG/heymax-analytics/blob/main/README.md>

About the Data

- event_time: timestamp, needs to be truncated to month
- user_id: string, uniquely identifies user
- event_type: e.g., miles_earned, miles_redeemed, share
- transaction_category: dining, ecommerce, flight, etc.
- miles_amount: some events may not have this (e.g. share)
- platform, utm_source, country: standard metadata fields

Technology Stack

DBT

DBT enables SQL-based transformation with support for testing, documentation, and scalable DAGs. Use '**dbt docs generate**' to view model docs. Parquet storage, partitioning, and sort keys improve performance in DuckDB.

DuckDB

DuckDB is an OLAP SQL engine ideal for analytical queries. It supports in-memory operations, Parquet IO, and integrates with cloud environments. Query via CLI (**'duckdb - ui'**) or embed in Python.

Streamlit

Streamlit builds interactive Python dashboards rapidly. It supports Python-native components, real-time reactivity, and deploys easily on Streamlit Cloud.

GitHub Deployment

- CI: Executes dbt runs/tests on every commit, emails logs, updates DuckDB, and runs daily at 8am. (the runs can be configured based on the requirements)
- CD: Uses Streamlit Cloud linked to the repo for zero-touch deployments.

Design Considerations

1. Data Modeling and Transformation Logic

Raw events are loaded from CSV and staged via ``stg_raw_events``. Business logic lives in intermediate models like ``dim_users``, ``fct_events``, and ``user_lifecycle_*`, leveraging incremental materialization and delete+insert logic. Partitioning and sort keys enhance query speed.

2. Dashboard Design and Metric Clarity

The dashboard offers toggles for daily/weekly/monthly views, chart summaries, cohort tables, and KPIs. Users can filter by date and export CSVs. LLM integration allows natural language queries.

3. Code Readability, Structure, and Documentation

SQL follows layered foldering (staging, intermediate, growth). Python code is modular with tabs. Metadata is defined via ``schema.yml``. CI/CD pipelines auto-generate docs and enforce quality.

4. Design Tradeoffs, Testing, and Scaling Strategy

DuckDB offers blazing speed for local analytics but isn't built for high-concurrency and access based control. Dbt tests ensure data quality. The modular structure supports easy migration to BigQuery or Snowflake.

DBT Modeling Strategy

This project uses a layered dbt modeling approach based on the Medallion Architecture. It transforms raw event data into actionable insights across daily, weekly, and monthly levels.

Model Summary

Model Name	Layer	Materialization	Strategy	Storage Format
stg_raw_events	Staging	Table	Full refresh	Parquet
stg_events	Staging	Table	Full refresh	Parquet
dim_users	Intermediate	Incremental	delete+insert	Parquet
fct_events	Intermediate	Incremental	delete+insert	Parquet
user_lifecycle_*	Intermediate	Incremental	delete+insert	Parquet
growth_metrics_*	Growth	View	-	-
retention_triangle_*	Growth	View	-	-

Layering and Its Benefits

This dbt project follows the Medallion Architecture:

1. Staging Layer
 - Purpose: Clean raw source data
 - Tables: stg_raw_events, stg_events
 - Characteristics: Simple SELECT + CAST, no business logic
 - Why Tables? DuckDB benefits from persisted Parquet tables for fast reads.
2. Intermediate (Core) Layer
 - Purpose: Central logic and transformations (fact/dim)
 - Tables: dim_users, fct_events, user_lifecycle_*
 - Why Incremental? For scalability and pipeline efficiency
 - Why delete+insert? Ensures upserts during recomputation
3. Growth Layer
 - Purpose: Analytical outputs and aggregations
 - Views: growth_metrics_*, retention_triangle_*
 - Why Views? These depend on constantly changing input and should reflect live business metrics

Analysis and Insights

Assumptions

- Each row in the source event data represents a unique user activity.
- Users are uniquely identified by `user\_id`.
- The `event\_time` field is the basis for all time-based aggregations (daily/weekly/monthly).
- Retention is calculated based on activity in consecutive periods post-cohort.
- Metrics are aggregated at the user level, not event count level.
- Churned users are defined as those who were active in the previous period but not in the current one.

Metric Definitions

- New Users: Users active for the first time in the selected period.
- Resurrected Users: Users who were inactive in the previous period but became active again.
- Retained Users: Users active in both the current and previous period.
- Churned Users: Users who were active previously but inactive in the current period.
- Active Users: Total users who performed at least one action in the period.
- Retention Rate: Percentage of retained users from the previous period.
- Quick Ratio: $(\text{New} + \text{Resurrected}) / \text{Churned}$ — indicates growth vs loss balance.

Insights

- There is a noticeable decline in new user acquisition beginning in March 2025, observed in both monthly and weekly data. No new users were recorded in the monthly dataset after March, and weekly data shows a complete halt starting the week of March 23. This suggests potential issues with acquisition strategies, marketing gaps, or external market shifts.
- Despite the drop in new users, retention rates remained strong in a few segments. Weekly cohorts from March 9 and 16 maintained 100% retention, and daily cohorts consistently showed full retention on day 0. This highlights that users who were acquired during that period were highly engaged initially.
- Retention tends to fall off rapidly across all periods. For example, the monthly March cohort dropped to 0% retention by month four, while daily cohorts often lose a significant portion of users after the first or second day. This points to a need for stronger post-onboarding engagement strategies.
- Daily metrics consistently show a quick ratio greater than 1, indicating that user growth (new + resurrected) often outpaces churn. This is a positive signal for user health during active periods, but it must be sustained with ongoing acquisition.
- Daily data reveals irregular acquisition behavior—significant spikes (e.g., 52 new users on March 8) followed by prolonged inactivity (e.g., no new users from June 1 onward). This inconsistency underscores the importance of establishing reliable, scalable growth channels.

Follow-up Questions

- What changes occurred around March 2025 in marketing campaigns, product launches, pricing, or external factors that might explain the observed decline in user acquisition?
- What characteristics define high-retention cohorts, such as those seen in the weekly and daily segments from early March? Are there acquisition sources, features, or behaviors that correlate with these outcomes?
- Are strategies currently in place to simultaneously support acquisition and retention? If so, how are they performing and being iterated?
- What improvements can be made to convert initial engagement (e.g., day 0 or week 1 activity) into longer-term usage, such as through onboarding flows, lifecycle messaging, or product nudges?
- Given the sporadic nature of daily acquisition spikes, what mechanisms (e.g., referrals, targeted outreach, event-based campaigns) could help replicate and sustain those surges?

Future Enhancements and Scaling Roadmap

Upgrade Area	Suggested Tools	Benefit
Data Warehouse / Lakehouse	Redshift , BigQuery, Snowflake, Databricks	Scales to handle large datasets, high concurrency, and complex analytics
Data Orchestration	Airflow, dbt Cloud, Dagster	Automates dbt runs, testing, dependency management, and alerting
Dashboarding	Looker, Tableau, Metabase	Enables production-ready dashboards with fine-grained role-based access control (RBAC)
Streaming Ingestion	Kafka, Pub/Sub	Supports near real-time data processing and dashboard updates

Additional Enhancements to current dashboard

- Add **granular UTM tracking, filters for country** for better user acquisition insights
- Have a **custom period** for churn, retention, resurrected and retained users based on the requirements of business.
- Implement **role-based access and authentication** in the Streamlit app using streamlit-authenticator
- Enhance the LLM interface with **chat history and intelligent prompt suggestions**